

Reviewer Pack — Quadratic Form Solution Count Exponential Gap in ACC⁰ Sipser...

Entry fb46b3cd2380 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 17:00:31 UTC

Quadratic Form Solution Count Exponential Gap in ACC⁰ Sipser Circuits

Entry ID: fb46b3cd2380 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 17:00:31 UTC

1. Conjecture

Field A (mathematical branch): Quadratic Forms over Finite Fields **Field B** (complexity object): ACC⁰ Circuit Complexity for Sipser Function

Statement:

For the Sipser function on n variables, the number of solutions to a system of quadratic forms derived from its truth table is $\Omega(2^{\{n/2\}})$, and any ACC⁰ circuit computing it must have size $\Omega(2^{\{n/2\}})$.

Rationale (proposer's reasoning):

Quadratic forms over finite fields encode parity constraints central to the Sipser function's structure. Their solution counts reveal exponential complexity barriers, which ACC⁰ circuits cannot overcome due to limited depth and modular arithmetic capabilities.

Taxonomy category: ACC_SIPSER (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b1332d993d1795b0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            return None # Singular matrix, no unique solution
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
    return A

def count_solutions(quadratic_system):
    A = quadratic_system
    n = len(A)
    solutions_count = 0
    try:
        reduced_matrix = gaussian_elimination(A)
        if reduced_matrix is None:
            return 0 # Singular matrix, no solution
        for i in range(n):
            if all(reduced_matrix[i][j] == 0 for j in range(i)) and reduced_matrix[i][i] != 1:
                solutions_count = 0
                break
            elif all(reduced_matrix[i][j] == 0 for j in range(i + 1, n)):
                solutions_count *= 2
    except IndexError:
        return 0 # Handle potential index errors during elimination
    return solutions_count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    truth_table = [[random.choice([0, 1]) for _ in range(n)] for _ in range(2**n)]

    quadratic_system = []
    for i in range(len(truth_table)):
```

```

        row = [truth_table[i][j] ^ truth_table[i][k] for j in range(k) for k in range(j + 1, n)]
        if any(row[j] != 0 for j in range(n)):
            quadratic_system.append(row)

    solutions_count = count_solutions(quadratic_system)
    conjecture_holds = solutions_count >= 2**(n // 2 - 5)
    counterexample = "" if conjecture_holds else f"Insufficient solution count: {solutions_count}"

    return {
        "metric_name": "solution_count",
        "metric_value": solutions_count,
        "instances_tested": len(truth_table),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_solution_count = sum(r["metric_value"] for r in results) / len(results)
    std_solution_count = math.sqrt(sum((r["metric_value"] - mean_solution_count) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_solution_count} std={std_solution_count} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_solution_count} std={std_solution_count} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Insufficient solution count\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Retry test with extended timeout and increased computational resources

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111213
2	propose	ollama_remote	qwen3:8b	0	0	35825
3	preregistration	ollama_remote	qwen3:8b	0	0	24058
4	novelty	ollama_remote	qwen3:8b	0	0	20717
5	novelty	ollama_remote	qwen3:8b	0	0	14496
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17821
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9466
8	judge	ollama_remote	qwen3:8b	0	0	85223

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 318819 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/fb46b3cd2380.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fb46b3cd2380.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fb46b3cd2380.tar.gz` (if generated)